AULA 19

Observer no Godot: sinais

Programação para Jogos I

Roteiro

1. Do Pong em Python ao Pong no Godot
2. A cena
3. Colisão no Godot
4. Sinais são o Observer
5. Demonstração
6. Os eventos do seu jogo

O que a engine já faz que, no Pong em Python, escrevemos à mão?

O que a engine já faz

No Python, à mão	No Godot
o game loop	o laço da engine
o passo fixo com acumulador	_physics_process a 60 Hz
percorrer as entidades	a árvore de cena
desenhar e trocar o quadro	cada nó se desenha
detectar a colisão	area_entered
a classe Sujeito	signal

O mesmo Pong, as mesmas peças

`World` o nó raiz da cena

`Entidade + Controle` raquete.tscn com um nó filho Controle

`colliderect()` Area2D com CollisionShape2D

`PASSO = 1/60` _physics_process(delta), a 60 Hz

Roteiro

1. Do Pong em Python ao Pong no Godot
2. A cena
3. Colisão no Godot
4. Sinais são o Observer
5. Demonstração
6. Os eventos do seu jogo

A árvore de nós do Pong

World

o jogo

- Jogador (raquete.tscn)
- Oponente (raquete.tscn)
- Bola
- GolEsquerdo
- GolDireito
- ParedeCima
- ParedeBaixo

quem escuta

- Placar
- Conquistas
  - nove nós
- Avisos

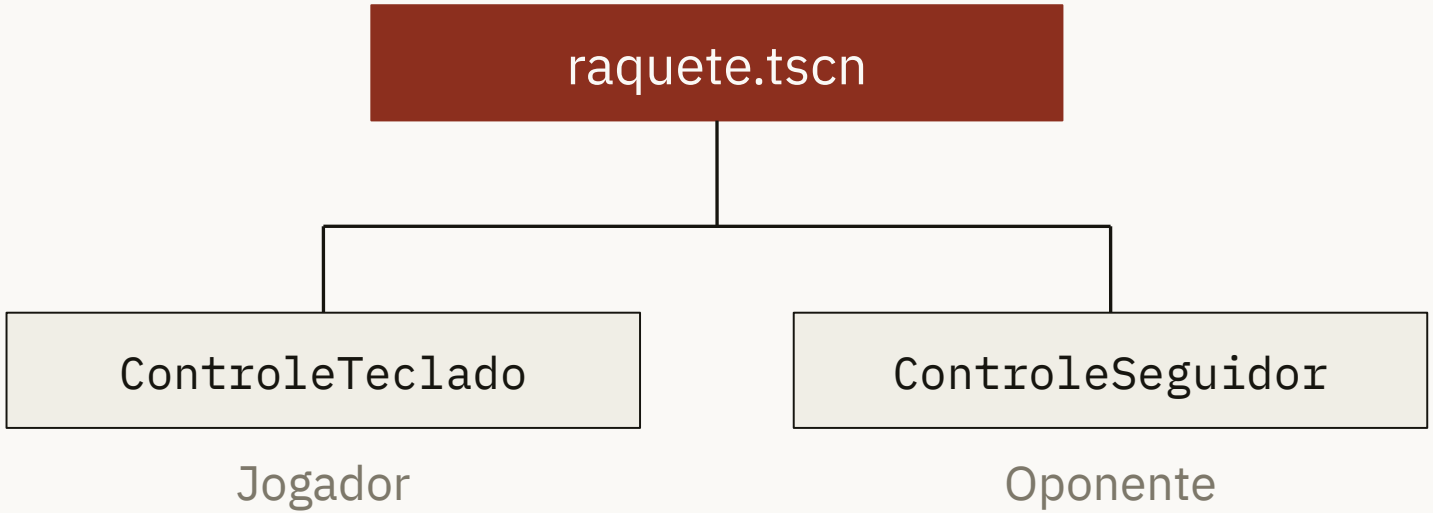
Uma pasta por peça, um script por responsabilidade

```
scripts/
```

`avisos/  bola/  gol/`	um script cada
`mundo/  placar/`	um script cada
`conquistas/`	seis scripts
`raquete/`	a raquete e os dois controles

O 18_pong_conquistas.py era um arquivo só.

Uma raquete, dois controles



Roteiro

1. Do Pong em Python ao Pong no Godot
2. A cena
3. Colisão no Godot
4. Sinais são o Observer
5. Demonstração
6. Os eventos do seu jogo

Um colisor é uma forma dentro de um corpo

`CollisionShape2D` a forma: onde o objeto ocupa espaço

`o nó pai` o corpo: o que acontece no contato

A mesma forma num Area2D só detecta; num StaticBody2D, bloqueia.

As formas são os volumes da Aula 08

`CircleShape2D` a bola

`RectangleShape2D` raquetes, gols e paredes

`CapsuleShape2D` personagens, como o Zé

`ConvexPolygonShape2D` o fecho convexo

Quatro tipos de corpo

Corpo	Quem move	No contato	Exemplos
`Area2D`	o código, ou nada	só detecta	gol, moeda, gatilho
`StaticBody2D`	ninguém	bloqueia	parede, chão
`CharacterBody2D`	o código	bloqueia e desliza	jogador, o Zé
`RigidBody2D`	a física	quica, empurra, gira	caixa, bola de bilhar

Por que, neste Pong, tudo é Area2D?

Neste Pong, a colisão só avisa

Movimento à mão, em _physics_process, como no Python

Rebote o World decide, na raquete e na parede

Contato a engine só avisa, com area_entered

E se a bola fosse um RigidBody2D?

Bola como RigidBody2D: ganha e perde

Ganha	Perde
rebote sem código	controle do ângulo
paredes viram StaticBody2D	velocidade previsível
giro e atrito, se quiser	a regra do rebote no nosso código

Camadas e máscaras: quem enxerga quem

`collision_layer` em que camadas o corpo está

`collision_mask` que camadas ele enxerga

No Pong: a bola na camada bola; raquetes, gols e paredes só enxergam a bola.

Quem percebe a colisão é a engine

`area_entered` sinal que a engine emite quando outra área entra

No Python, colisoes() testava collideRect() a cada passo. Aqui ninguém testa: a engine avisa.

Se a engine já avisa a colisão, o que sobra para o World?

Roteiro

1. Do Pong em Python ao Pong no Godot
2. A cena
3. Colisão no Godot
4. Sinais são o Observer
5. Demonstração
6. Os eventos do seu jogo

Um sinal é um evento declarado no nó

```
signal ponto(autor)
signal rebatida(autor)
signal saque()
```

Declarados no World. O World é o sujeito.

O sinal já traz a lista de observadores

`connect()` põe um método na lista do sinal

`emit()` chama cada método da lista

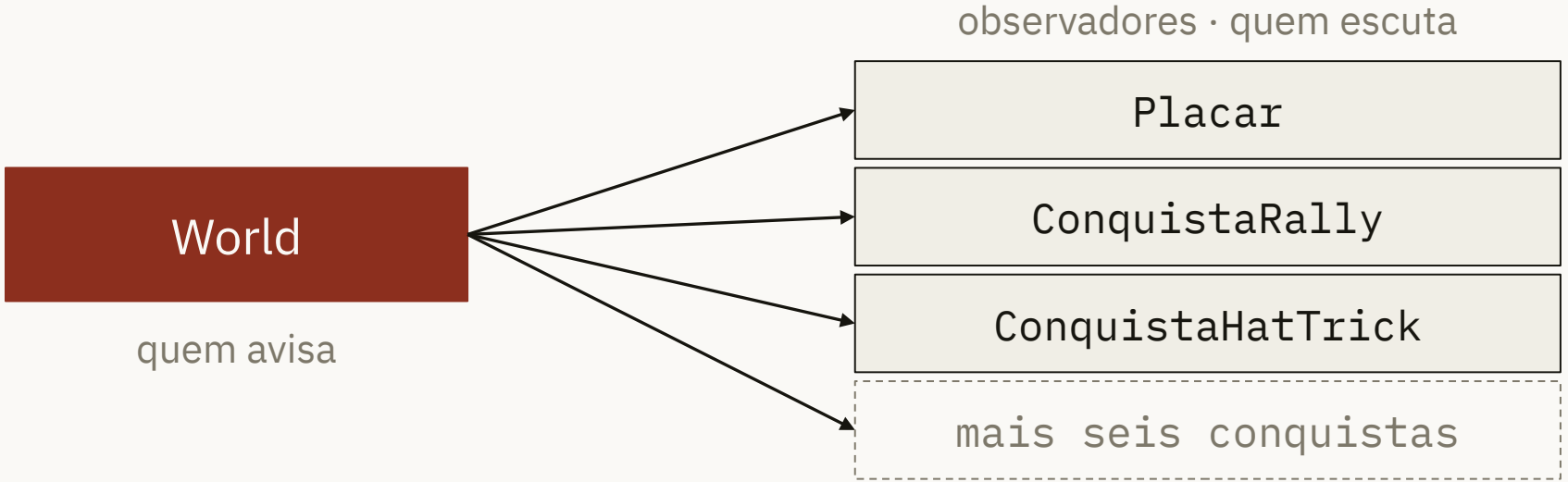
`disconnect()` tira um método da lista

É o Sujeito da Aula 18, pronto, dentro de cada sinal.

Aula 18 e Godot, lado a lado

Aula 18 (Python)	Godot
Sujeito	nó que declara signal
notificar("ponto", autor)	ponto.emit(autor)
inscrever(observador)	ponto.connect(placar._on_ponto)
desinscrever(observador)	ponto.disconnect(_on_ponto)
ao_notificar(evento, autor)	_on_ponto(autor)

ponto.emit() avisa todos os conectados




```
ponto.emit("oponente")
```

Cada um recebe o mesmo aviso e decide o que fazer.

O World percebe e avisa

Percebe		Reage
bola entrou na raquete	→	ajusta a direção, rebatida.emit(autor)
bola entrou no gol	→	ponto.emit(autor), repõe a bola
bola entrou na parede	→	inverte vy, sem sinal
bola reposta no centro	→	saque.emit()

O World não chama placar nem conquistas.

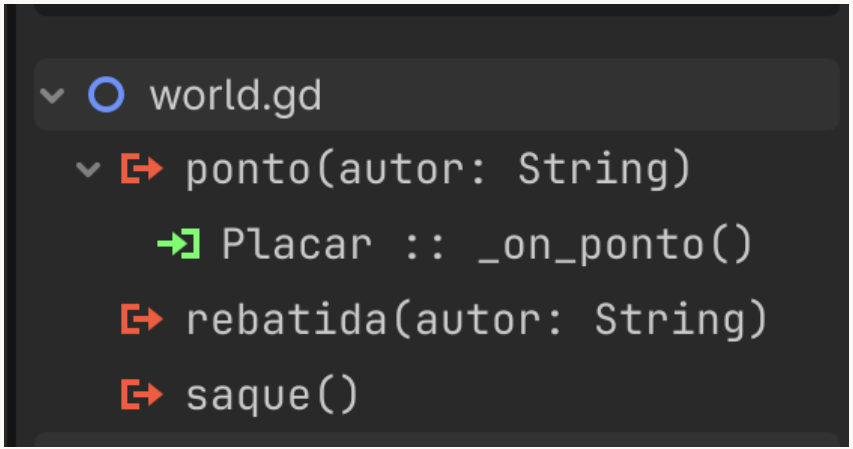
No Python, ao_notificar testava o evento com if. Por que aqui não precisa?

Duas formas de conectar

Pelo editor aba Nó → Sinais; a conexão fica salva na cena

Por código connect() no _ready(), num laço sobre as conquistas

A aba Nó → Sinais



vermelho sinal que o World emite

verde conexão feita no editor

vazio rebatida e saque: conectados por código

A aba mostra só as conexões do editor. As do código só existem com o jogo rodando.

Roteiro

1. Do Pong em Python ao Pong no Godot
2. A cena
3. Colisão no Godot
4. Sinais são o Observer
5. Demonstração
6. Os eventos do seu jogo

Nove conquistas, seis classes

conquista_...	Escuta	Nós
primeiro_ponto	ponto	1, e se desconecta
rebatidas_seguidas	rebatida, ponto	1
rally	rebatida, ponto	1
pontos_seguidos	ponto	2: hat-trick, levou 3
ponto_sem_toque	saque, rebatida, ponto	2: boa e ruim
virada	ponto	2: boa e ruim

Uma classe, duas conquistas

`ConquistaHatTrick` jogador · 3

`ConquistaLevouTresSeguidos` oponente · 3 · ruim

Mesmo script, configurado no Inspetor: quem, quantidade, texto, ruim.

Desligar conquistas: mexer na árvore

Tirar um nó de Conquistas aquela conquista some

Tirar o nó Conquistas modo treino: nenhuma conquista

O jogo roda igual. world.gd não muda.

Roteiro

1. Do Pong em Python ao Pong no Godot
2. A cena
3. Colisão no Godot
4. Sinais são o Observer
5. Demonstração
6. Os eventos do seu jogo

Os eventos do seu jogo

1. Que eventos acontecem no seu jogo?
2. Quem emite cada um?
3. Quem precisa escutar cada um?
4. Algum deles seria melhor como chamada direta, sem sinal?

Vai para o SGDD, que entrega sexta.

Se o placar estivesse em outra cena, a quem ele se conectaria?